

11月7日实验：服务通信和动作通信

一、服务通信

服务通信代码实现

1. 自定义通信接口

- i. 创建C++功能包my_interfaces用于编写接口文件（或者直接使用之前自定义话题接口时创建的功能包）
- ii. 在my_interfaces/srv目录中编写服务接口srv文件

示例：AddTwoInts.srv

```
# 请求的数据结构
int64 num1
int64 num2

#分割线
---

# 响应的数据结构
int64 sum
```

iii. 在package.xml添加依赖

```
<build_depend>roslaunch</build_depend>
<exec_depend>roslaunch</exec_depend>
<member_of_group>roslaunch</member_of_group>
```

iv. 配置CMakeLists.txt

```
find_package(roslaunch REQUIRED)

roslaunch_generate_interfaces(${PROJECT_NAME}
"srv/Add.srv"
DEPENDENCIES std_msgs
)
```

- v. 编译功能包，并使用ros2 interface检查是否成功
- vi. 回到要使用该接口的功能包进行使用

2. 服务端实现示例

核心代码

```
from my_interfaces.srv import AddTwoInts

class Service_Server(Node):
    def __init__(self,name):
        super().__init__(name)
        #创建一个服务端，使用的是create_service函数
        #参数：服务数据的数据类型、服务的名称，服务回调函数（也就是服务的内容）
        self.srv = self.create_service(AddTwoInts, '/add_two_ints', self.Add2Ints_callback)
        #服务回调函数。request是服务需要的参数，response是服务的反馈结果
    def Add2Ints_callback(self,request,response):
        response.sum = request.a + request.b
        print("response.sum = ",response.sum)
        return response
```

3. 客户端实现示例

核心代码

```
# 节点定义
class Service_Client(Node):
    def __init__(self,name):
        super().__init__(name)
        #创建客户端，使用的是create_client函数，传入的参数是服务数据的数据类型、服务的话题名称
        self.client = self.create_client(AddTwoInts,'/add_two_ints')
        # 循环等待服务器端成功启动
        while not self.client.wait_for_service(timeout_sec=1.0):
            print("service not available, waiting again...")
        # 创建服务请求的数据对象
        self.request = AddTwoInts.Request()

    def send_request(self):
        self.request.a = 10
        self.request.b = 90
        #发送服务请求
        self.future = self.client.call_async(self.request)
```

```
# main函数中
while rclpy.ok():
    rclpy.spin_once(service_client)
    #判断数据是否处理完成
    if service_client.future.done():
        try:
            #获得服务反馈的信息并且打印
            response = service_client.future.result()
            print("Result = ",response.sum)
        except Exception as e:
            service_client.get_logger().info('Service call failed %r' % (e,))
    break
```

二、动作通信

动作通信代码实现

1. 自定义动作接口

- i. 在接口功能包目录下新建action文件夹，在action文件夹下创建Progress.action文件
- ii. 文件内容示例:

```
# request
int64 num
---
# respond
int64 sum
---
# feedback
float64 progress
```

- iii. 在package.xml中需要添加一些依赖包

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<depend>action_msgs</depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

iv. 在CMakeLists.txt 中添加如下配置

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Progress.action"
)
```

v. 编译功能包，检查是否成功，再使用接口

2. 服务端实现示例

核心代码

```
from rclpy.action import ActionServer
#导入我们编写的通信接口
from pkg_interfaces.action import Progress

class Action_Server(Node):

    def __init__(self):
        super().__init__('progress_action_server')
        # 创建动作服务端
        self._action_server = ActionServer(
            self,
            Progress,
            'get_sum',
            self.execute_callback)

    def execute_callback(self, goal_handle):

        # 生成连续反馈；
        feedback_msg = Progress.Feedback()

        sum = 0
        for i in range(1, goal_handle.request.num + 1):
            sum += i
            feedback_msg.progress = i / goal_handle.request.num
            goal_handle.publish_feedback(feedback_msg)
            time.sleep(1)

        # 生成最终响应。
        goal_handle.succeed()
        result = Progress.Result()
        result.sum = sum

        return result
```

3. 客户端实现示例

核心代码

```
class Action_Client(Node):
    def __init__(self):
        super().__init__('progress_action_client')
        # 创建动作客户端：
        self._action_client = ActionClient(self, Progress, 'get_sum')

    def send_goal(self, num):
        # 发送请求；
        goal_msg = Progress.Goal()
        goal_msg.num = num
        self._action_client.wait_for_server()
        self._send_goal_future = self._action_client.send_goal_async(goal_msg, feedback_callback=self.feedback_callback)
        self._send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        # 处理目标发送后的反馈；
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().info('请求被拒绝')
            return

        self.get_logger().info('请求被接收，开始执行任务！')

        self._get_result_future = goal_handle.get_result_async()
        self._get_result_future.add_done_callback(self.get_result_callback)

    # 处理最终响应。
    def get_result_callback(self, future):
        result = future.result().result
        self.get_logger().info('最终计算结果:sum = %d' % result.sum)
        # 5.释放资源。
        rclpy.shutdown()

    # 处理连续反馈；
    def feedback_callback(self, feedback_msg):
        feedback = (int)(feedback_msg.feedback.progress * 100)
        self.get_logger().info('当前进度: %d%' % feedback)
```

```
def main(args=None):
    rclpy.init(args=args)
    action_client = Action_Client()
    action_client.send_goal(10)
    rclpy.spin(action_client)
```

三、实验要求

请实现以下节点，通过各节点之间协作模拟小车在二维平面内执行巡逻任务，巡逻期间期间用户可以随时在**自动巡逻**和**手动操控**之间切换小车的行动模式。**自动巡逻**模式下，小车总是向着下一个还未到达的巡逻目标点匀速直线前进；**手动遥控**下，小车朝着用户操作的方向（上下左右停）匀速直线运动。

1. 小车模拟节点：

- 动作通信服务端。接收**巡逻点设置请求**，接收用户指定的一系列巡逻点的二维坐标并进行保存。在收到**巡逻点设置请求**后，小车立即进入**自动巡逻**模式，每当抵达一个目标点时返回该目标点的feedback，所有巡逻点都依次抵达后返回result。若用户切换到**手动遥控**模式，则中断巡逻任务，根据订阅到的遥控信息进行移动，直到用户再切换回**自动巡逻**模式（手动遥控期间不考虑抵达巡逻点）。每1秒将小车的二维坐标、行动模式以及移动总距离打印到控制台。
- 服务通信服务端。接收**模式切换请求**，在**自动巡逻**模式和**手动遥控**模式间切换，返回response表示是否切换成功。

- 话题通信订阅者。订阅**遥控操作**，接收用户最近指定的方向并进行保存。

2. 巡逻点设置节点：

- 动作通信客户端。设置小车需要经过的巡逻点，将收到的feedback和result打印到控制台。

```
# my_interface/action/Patrol.action

# Goal: 巡逻目标点列表
geometry_msgs/Point[] points
---

# Result
bool success

---

# Feedback: 抵达的目标点信息
geometry_msgs/Point point_arrived
```

除了前面提到的自定义接口时需要添加的配置外，使用**geometry_msgs**额外需要的配置如下：

```
# package.xml
<depend>geometry_msgs</depend>

# CMakeLists.txt
find_package(geometry_msgs REQUIRED)
rosidl_generate_interfaces(
  ${PROJECT_NAME}
  "action/Patrol.action"
  DEPENDENCIES geometry_msgs
)
ament_export_dependencies(rosidl_default_runtime geometry_msgs)
```

3. 模式切换节点：

- 服务通信客户端。根据输入切换小车的移动模式，将收到的response打印到控制台。

```
# my_interface/srv/SwitchMode
bool keyboard_control

---

string message
```

4. 手动遥控节点：

- 话题通信发布者。根据用户输入，发布小车移动方向信息以操控小车。

其他注意点：小车视作二维平面内的一个点即可，不用考虑小车运动的角速度和车头方向以外的线速度。geometry_msgs/Point中包含三维信息，本次不需要使用到z轴信息，z设置为0即可。小车具体的行进速度、设置的巡逻目标点等可以自行设定。

提交要求

提交实验报告（包含简单的实现思路和结果截图）以及实验代码（包含整个工作空间）